

SimpleGame

Документация проекта

Двумерная tower defense на Java 17 и LibGDX 1.14.2. Общая логика живёт в модуле core, десктопный запуск — в lwjgl3, мобильная сборка — в android.

Версия проекта: 1.0.0. Документ составлен по исходному коду пакета io.github.simple_game.core (сентябрь 2026).

Содержание

1. Обзор
2. Сборка и запуск
3. Архитектура слоёв
4. Игровой мир и сессия
5. Башни и снаряды
6. Враги и волны
7. Экономика, магазин, инвентарь
8. Презентация: экраны, ввод, рендер
9. Расширение игры
10. Текущие ограничения

1. Обзор и стек

SimpleGame — tower defense с фиксированной сеткой 10×16 клеток. Игрок ставит башни с дороги, переживает 20 волн и может экипировать предметы, выпавшие с врагов. Инвентарь общий для сессий: Main хранит InventoryManager, а каждая партия создаёт свой GameLoop.

Модули Gradle

Модуль	Роль
core	Вся игровая логика, модель, сервисы и UI
lwjgl3	Десктоп: Lwjgl3Launcher, окно LWJGL3
android	Android: applicationId io.github.simple_game, minSdk 21, targetSdk 34

Технологии

Компонент	Значение
Язык	Java 17
Движок	LibGDX 1.14.2
Сборка	Gradle 9.5.1 (wrapper), Android Gradle Plugin 8.9.3
LWJGL	3.4.1
Версия приложения	1.0.0 / versionName 1.0

Ресурсы лежат в корневом каталоге assets/: тайлы карты, спрайты башен и врагов, карточки магазина, иконки UI и предметов. Список файлов при сборке пишется в assets/assets.txt задачей generateAssetList.

2. Сборка и запуск

Команды выполняются из корня репозитория. На Windows используйте gradlew.bat.

```
gradlew.bat lwjgl3:run
gradlew.bat :core:compileJava
gradlew.bat android:installDebug android:run
gradlew.bat clean assembleDebug
```

- Десктопный main-класс: io.github.simple_game.lwjgl3.Lwjgl3Launcher. Рабочая директория run — папка assets.
- Логический размер UI — 480×800 (GameViewport). Игровое поле — 480×768 (10×16 клеток по 48 px). Камера мира — FitViewport, UI — ExtendViewport.

- В `gradle.properties` логирование Gradle выставлено в `quiet`, демон отключён.

3. Архитектура слоёв

Зависимости направлены только вниз: `Main` → `presentation` → `service` → `model`. Модель не импортирует `GameLoop`, `CurrencyManager` и `GameViewport`. Текстуры и анимации загружает презентация.

Main

`io.github.simple_game.core.Main` наследует `com.badlogic.gdx.Game`. При `create()` создаёт глобальный `InventoryManager` и открывает `MainMenuScreen`. Запасной `GameLoop` больше не строится.

model

- `CombatWorld` — узкий контракт для башен: списки башен, врагов и экипированных предметов. Реализует `GameLoop`.
- `entity`: `Entity`, враги, башни, снаряды, предметы, `GameGrid`.
- `movement`: `RoadPath` (вейпоинты + ID вариантов прямой дороги), `PathGenerator`, `WalkMovement`, `PathType`.

service

- `GameLoop` — фасад сессии: сетка, волны, экономика, магазин, сущности, пауза, победа.
- `EntityManager`, `WaveManager`, `EnemyFactory`, `CurrencyManager`, `ShopService`, `InventoryManager`.

presentation

- `screen`: `MainMenuScreen`, `GameScreen`, `InventoryScreen`.
- `input`: `CameraGestureService`, `InteractionService`, `DragAndDropManager`.
- `ui`: статус-бар, магазин, апгрейд, окна победы/поражения, инвентарь.
- `view/renderer`: `GameRenderer`, `WorldSpriteRenderer`, `EntityRenderer`, `HealthBarRenderer`, `DebugGridRenderer`, `EnemySprites`, `TowerSprites`.

Башни получают `CombatWorld`, а не весь цикл. Проверка клетки: `GameGrid.isCellBuildable(x, y, towers)`. Золото и жизни начисляются только в `EntityManager` при удалении врага: `health <= 0` — награда и лут, иначе `-1` жизнь.

4. Игровой мир и сессия

Сетка

`GameGrid.CELL_SIZE` = 48, `COLS` = 10, `ROWS` = 16. Мир 480×768. `snap()` центрирует координату в клетке. На дорогу и ближе чем 1.1 клетки от другой башни строить нельзя.

Дорога

При создании `GameLoop` `PathGenerator.generate` выбирает случайный `PathType`: `STRAIGHT_FEW_TURNS`, `MANY_TURNS` или `WITH_LOOPS`. Путь осе-выровненный, параллельные участки разделены минимум одной клеткой земли. Для прямых сегментов: 30% стандартный тайл, 70% один из трёх альтернативных (ID 1–3 на `RoadPath`). Рендерер подставляет PNG.

Старт сессии

Параметр	Значение
Стартовое золото	10 000
Жизни базы	20
Условие победы	Волна 20 очищена, врагов на карте нет
Пауза	<code>GameLoop.isPaused</code> — update не идёт

Каждый кадр `GameLoop.update` вызывает `WaveManager` и `EntityManager`. После смерти врага `EntityManager` либо добавляет золото и считает лут, либо списывает жизнь, если враг дошёл до конца (`onReachedEnd`, здоровье ещё больше нуля).

5. Башни и снаряды

Новый тип: класс-наследник `Tower` + константа в enum `TowerType` (цена, idle-текстура, скорость снаряда, фабрика). Карточка магазина: `card/<имя>_card.png`, анимация появления: `towers/<имя>_init.png`. После постановки башня ~0.48 с не стреляет (окно спавна); кадры рисует `TowerSprites`.

Тип	Цена	Урон	Радиус	КД, с	Снаряд
ARCHER	100	15	150	0.6	Arrow, 400, крит 20% ×2
CANNON	250	50	120	2.0	CannonBall, 250, взрыв R=70
MAGIC	200	10	130	1.2	MagicSphere, 320
POISON	175	5	140	1.5	PoisonBolt, 280 + DoT 8/4с
TESLA	150	12	200*	0.4	Нет снаряда, луч между парой

*У Tesla радиус — поиск партнёра. Одна колонна не бьёт; две в радиусе связи образуют молнию (ширина линии 10). Урон на линии наносит только «первичная» из пары, чтобы не бить дважды.

Улучшения

Максимум 5 уровней. Стоимость апгрейда: $(\text{int})(\text{type.cost} * 0.5 * \text{currentLevel})$. Лучник: урон $\times 1.15$, радиус $\times 1.05$, КД $\times 0.90$ за уровень, крит +5% за уровень. Пушка и магия масштабируют свои базы. Яд: урон и DPS $\times$ уровень, радиус +10 за уровень, длительность яда +0.5 с.

Поиск цели и снаряды

Башня держит первую цель в радиусе. Снаряды самонаводятся каждый кадр; если цель умерла, летят в последнюю известную точку. Попадание вызывает `takeDamage`. Яд вешает `applyPoison`. Пушка бьёт всех в радиусе 70 от точки взрыва.

6. Враги и волны

Класс	Тип	HP	Скорость	Золото	Спрайт
FastGoblin	TIER_1_LIGHT (вес 1)	50	130	15	enemies/goblin.png
NormalZombie	TIER_2_NORMAL (вес 2)	100	70	25	enemies/zombie.png
HeavyOrc	TIER_3_HEAVY (вес 4)	300	40	60	enemies/orc.png

EnemyFactory масштабирует волной: $\text{HP} \times (1 + (\text{wave}-1) \times 0.10)$, скорость $\times (1 + \min(0.4, (\text{wave}-1) \times 0.03))$. Движение — WalkMovement по вейпоинтам RoadPath, независимо от FPS (deltaTime). Есть замедление и яд. Поворот спрайта считается по смещению за кадр; анимацию бега кэширует EnemySprites.

Волны

Старт следующей волны — кнопка на верхней панели (WaveManager.startNextWave). Пока волна активна, повторный старт игнорируется. Бюджет: $4 + \text{номер_волны} \times 3$. С волны 3 с шансом 40% берётся NORMAL, с волны 5 с шансом 20% — HEAVY, иначе LIGHT. Очередь перемешивается, спавн раз в 1.0 с. Волна кончается, когда очередь пуста и на карте нет врагов.

7. Экономика, магазин, инвентарь

Магазин

ShopService держит 3 случайных разных TowerType. Обновление после покупки или за 20 золота (REFRESH_COST). Покупка: DragAndDropManager проверяет золото, снаряд клетки и isCellBuildable, затем `type.create(..., gameLoop)` и `spendGold`.

Предметы

Инвентарь живёт в Main и переживает рестарт партии. Три слота экипировки, три слота кузницы, рюкзак. Экипированные предметы каждый кадр вызывают `applyEffect` на башнях (через `CombatWorld.getEquippedItems`).

Предмет	Дроп	Эффект
Rusty Arrow	15%, TIER_1	Лучник: радиус +40
Mana Crystal	8%, TIER_2	Маг: КД $\times 0.8$
Heavy Core	10%, TIER_3	Пушка: урон +25
Spring Arrow	не дропается	Лучник: КД $\times 0.75$; крафт из 3× Rusty Arrow

С каждого убитого врага `InventoryManager.calculateLootDrop` проходит таблицу лута и кладёт в рюкзак первый предмет подходящего тира, если `random() <= dropChance` (не больше одного предмета за смерть). Кузница: три предмета; Spring Arrow узнаёт рецепт из трёх SharpArrow. Шанс крафта в коде сейчас 100%.

8. Презентация: экраны, ввод, рендер

Экраны

- `MainMenuScreen` — старт партии, переход в инвентарь и выход.
- `GameScreen` — создаёт `GameLoop(globalInventory)`, `FitViewport` мира, `ExtendViewport UI`, мультитекстурный ввод (`Stage` → жесты камеры → взаимодействие). При ресайзе вычитает высоту статус-бара и магазина, чтобы HUD не перекрывал поле.
- `InventoryScreen` — рюкзак, экипировка, кузница; `Scene2D DragAndDrop`.

Ввод

- `CameraGestureService` — pan, pinch-zoom, инерция, камера ограничена размером мира.
- `InteractionService` — выбор башни по тапу, прокси к `DragAndDropManager`.
- `DragAndDropManager` — превью постановки, проверка клетки и золота.

Рендер

`GameRenderer` рисует слоями: фон и дорога, башни (`idle` или `init`-анимация), отладочная сетка и радиусы при драге, враги и снаряды, HP-бары, призрак перетаскиваемой башни. UI — `GameInterface` (`TopStatusBar`, `TowerControlPanel`, `ShopPanel`, `GameOverWindow`, `VictoryWindow`). Колбэки экрана: открыть инвентарь, рестарт, выход в меню — виджеты не знают Main.

9. Расширение игры

Новая башня

- Класс в `model.entity.tower`, снаряд при необходимости в `projectile`.
- Константа `TowerType`: цена, `towers/<id>.png`, скорость снаряда, ссылка на конструктор.
- Аксеты: `idle`, `towers/<id>_init.png`, `card/<id>_card.png`.

Новый враг

- Подкласс `Enemy` + тип в `EnemyTier` (вес бюджета) + ветка в `EnemyFactory`.
- Спрайт-лист 32 px в `assets/enemies/`, путь в `getSpritePath()`.

Новый предмет

- Подкласс `Item`: имя, описание, шанс, тип дропа, `applyEffect`, `clonePrototype`.
- Зарегистрировать экземпляр в `lootTable` `InventoryManager`. Для крафта переопределить `checkRecipe`.
- Иконка: `items/<имя_в_нижнем_регистре_с_подчёркиваниями>.png`.

10. Текущие ограничения

- `GameLoop` остаётся широким фасадом: выбранная башня, пауза, победа, магазин, сетка и списки сущностей на одном типе. Рендереры и панели принимают весь объект.
- `RoadPath` хранит целочисленные ID вариантов прямой дороги — это визуальные данные в модели; PNG подставляет `WorldSpriteRenderer`.
- Стартовые 10 000 золота выглядят как отладочный баланс, не как финальная экономика.
- Крафт `Spring Arrow` всегда успешен (порог 1.00f).
- Модель использует `LibGDX Vector2` и `Array` — это математика движка, не инверсия слоёв.
- Сохранения прогресса нет: инвентарь живёт, пока жив процесс приложения.

Источник правды — код в `core/src/main/java`. Если баланс или пакеты изменятся, пересоберите PDF: `python docs/generate_documentation.py`